

Лекция 2. Системы управления базами данных и язык SQL

Дисциплина: Методы и средства проектирования баз данных

Составитель: Монастырев Егорий Егорьевич, М-ВТ-25

Якутск, 2026

Что должно стать понятным после лекции

- ▶ чем СУБД отличается от самой базы данных и какую роль она выполняет;
- ▶ из каких основных компонентов состоит СУБД и как проходит запрос;
- ▶ как клиентское приложение взаимодействует с сервером базы данных;
- ▶ что такое SQL и как его команды группируются по назначению;
- ▶ чем отличаются DDL, DML, DCL и TCL;
- ▶ как выглядят базовые SQL-операции, которые будут использованы в лабораторной работе в Microsoft Access 2016.

Логика лекции

От устройства СУБД → к языку SQL → к конкретным командам.

Что мы уже знаем из лекции 1

В лекции 1 мы рассмотрели предметную область, данные и основные модели их представления, ER-модель и переход от предметной области к структуре базы данных.



Новая точка фокуса

Не повторяем построение ER-модели и архитектуру ANSI/SPARC. Теперь рассматриваем, **как СУБД реально обслуживает операции над уже спроектированной БД.**

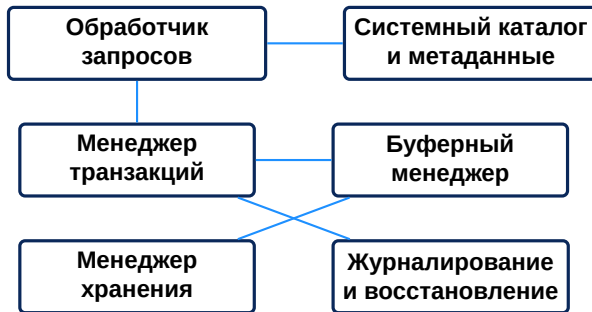
Определение

СУБД — программная система, предназначенная для определения структуры базы данных, хранения и обработки данных, управления доступом к ним, поддержания целостности, транзакционной обработки и восстановления после сбоев.

- ▶ приложение формулирует операцию;
- ▶ СУБД проверяет права и ограничения;
- ▶ СУБД выбирает способ выполнения операции;
- ▶ СУБД организует чтение или изменение данных;
- ▶ результат возвращается клиенту.

Функция	Задача
Хранение	размещение и чтение данных на внешней памяти
Обработка запросов	разбор, построение и выполнение плана запроса
Целостность	контроль ограничений и допустимых состояний данных
Транзакции	согласованное выполнение группы операций
Конкурентный доступ	корректная работа нескольких пользователей
Безопасность	аутентификация, роли и права доступа
Журналирование и восстановление	сохранение информации, необходимой для восстановления после сбоев

Из каких частей состоит СУБД



Не нужно запоминать реализацию конкретного продукта

Архитектура конкретной СУБД различается деталями, но набор решаемых задач имеет одну общую инженерную основу.



- ▶ разбор: синтаксис и смысловые элементы;
- ▶ планирование: какой способ выполнения выбрать;
- ▶ выполнение: чтение или изменение данных;
- ▶ возврат результата клиенту.

Клиент и ядро СУБД в Microsoft Access



Ключевая идея

В Access 2016 прикладная программа взаимодействует с ядром СУБД Access Database Engine, которое выполняет SQL-запросы и управляет файлами базы данных. Это отличается от классической серверной архитектуры PostgreSQL.

Кто взаимодействует с СУБД

Участник	Типичные задачи
Администратор БД	создание ролей, настройка доступа, резервное копирование, контроль эксплуатации
Разработчик	проектирование схемы, SQL, транзакции, работа с API СУБД
Аналитик / специалист	формирование запросов, выборка и анализ данных
Конечный пользователь	работа с данными через прикладное приложение

SQL

SQL (Structured Query Language) — стандартизованный язык работы с реляционными базами данных, включающий средства определения структуры, манипулирования данными и управления доступом и транзакциями.

- ▶ SQL в основном декларативен: описывается требуемый результат или операция;
- ▶ конкретный план выполнения выбирает СУБД;
- ▶ разные СУБД поддерживают SQL с диалектными и функциональными отличиями.

Важно для курса

SQL имеет стандартную основу, но Microsoft Access использует собственный диалект Access SQL. Поэтому некоторые типы данных, функции, операторы подстановки и отдельные конструкции отличаются от PostgreSQL, MySQL и других СУБД.

Задача	Access SQL	Пример
Шаблон строки	LIKE 'ИВ*'	фамилия начинается с «ИВ»
Дата AutoNumber	#2026-09-07# COUNTER	дата в условии запроса автоматически создаваемый идентификатор
Логика	AND, OR, NOT	составное условие

Подъязыки SQL: карта лекции

DDL
структура

DML
данные

DCL
доступ

TCL
транзакции

DDL	определение объектов БД	CREATE, ALTER, DROP
DML	изменение и выборка данных	SELECT, INSERT, UPDATE, DELETE
DCL	управление правами	GRANT, REVOKE
TCL	управление транзакциями	BEGIN TRANSACTION, COMMIT, ROLLBACK

Примечание: классификация команд в учебной литературе может отличаться; например, SELECT иногда выделяют отдельно как DQL.

Назначение

DDL используется для **определения и изменения объектов базы данных**: схем, таблиц, ограничений, индексов, представлений и других объектов.



Практический смысл

DDL отвечает на вопрос: **какие объекты должны существовать в БД и как они устроены?**

Зачем нужен тип данных?

Тип данных определяет, какие значения может хранить поле, какие операции над ними допустимы и как СУБД должна интерпретировать это значение.

Группа	Тип Access SQL	Пример применения
Числовые	BYTE, SMALLINT, INTEGER, REAL, FLOAT, DECIMAL	идентификаторы, количество, измерения
Строковые	TEXT, VARCHAR, CHAR	ФИО, коды, названия
Дата и время	DATETIME	дата рождения, дата события
Логические	BIT / YESNO	да/нет, активно/неактивно
Денежные	MONEY / CURRENCY	цены и суммы
Специальные	COUNTER, AUTOINCREMENT, UUID, LONGTEXT	AutoNumber, GUID, длинный текст

Основные числовые типы Access SQL

Тип	Смысл	Пример
BYTE	небольшое целое число	небольшой счётчик
SMALLINT	целое малой разрядности	номер курса
INTEGER	длинное целое число	идентификатор, количество
REAL	число с плавающей точкой одинарной точности	измерения
FLOAT	число с плавающей точкой двойной точности	вычисления
DECIMAL (p, s)	точное десятичное значение	значения с фиксированной точностью
MONEY/CURRENCY	денежные значения	цена, сумма

Важно для Access

В SQL ядра Access тип INTEGER соответствует длинному целому числу (Long Integer) поля Access. Это отличается от привычного сопоставления типов в некоторых других СУБД.

Строковые, логические и временные типы Access

Тип SQL	Пример	Применение
TEXT(n) / VARCHAR(n)	'Иванов Иван'	короткий и средний текст
MEMO / LONGTEXT	длинное описание	большие текстовые поля
BIT / YESNO	TRUE/FALSE	логический признак
DATETIME	07.09.2026 21:00	дата и время события
UNIQUEIDENTIFIER / GUID	GUID	глобально уникальный идентификатор

Практическое правило

Не хранить даты, числа и логические признаки как обычный текст, если над ними нужно выполнять соответствующие операции сравнения, сортировки или вычисления.

Тип данных и свойства поля в Access

В Access 2016 тип данных — только одна часть определения поля. В конструкторе таблицы с ним связаны дополнительные свойства.

Свойство	Что задаёт
Тип данных	множество допустимых значений
Размер поля	объём/размер для некоторых типов
Обязательное поле	запрещает отсутствие значения
Индексированное	создаёт индекс для ускорения поиска
Значение по умолчанию	значение, подставляемое при добавлении записи
Правило проверки	условие допустимости значения

Главная мысль

Тип данных и ограничение поля — разные вещи. Например, `INTEGER` не означает автоматически «это первичный ключ».

Как выбрать тип данных в Access 2016

1. Определить **смысл** значения.
2. Выбрать наиболее подходящий тип Access SQL.
3. Для идентификатора, который должен автоматически создаваться, использовать COUNTER / поле «Счётчик».
4. Для даты использовать DATETIME, а не текстовое поле.
5. Для денежных величин рассматривать MONEY/CURRENCY.
6. Правила обязательности, уникальности и допустимого диапазона задавать отдельно.

Пример

name TEXT — имя; course INTEGER — номер курса; birth_date DATETIME — дата рождения; active BIT — признак активности.

NULL: отсутствие значения — не тип данных

Что такое NULL?

NULL обозначает отсутствие известного или заданного значения. Это не строка 'NULL', не число 0 и не пустая строка ''.

Значение	Пример	Смысл
Число	0	известное числовое значение
Пустая строка	''	известная строка длины 0
NULL	NULL	значение отсутствует / неизвестно

Проверка отсутствия значения выполняется через `IS NULL` / `IS NOT NULL`, а не через `= NULL`.

Идентификаторы: AutoNumber и COUNTER

В Microsoft Access 2016 для автоматически создаваемого идентификатора используется поле типа **Счётчик (AutoNumber)**; в SQL Access ему соответствует COUNTER (синоним AUTOINCREMENT).

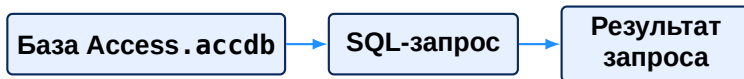
```
CREATE TABLE Student (  
    ID COUNTER CONSTRAINT PK_Student PRIMARY KEY,  
    Name TEXT(100) NOT NULL  
);
```

Важно

В отличие от PostgreSQL, в лабораторной работе мы не используем SERIAL или GENERATED ... AS IDENTITY. Эти конструкции относятся к другим СУБД.

Что будет на практике

В лабораторной работе SQL-запросы будут создаваться и выполняться в **Microsoft Access 2016**. Мы будем работать с базой данных Access и окном запроса в режиме SQL, а не с PostgreSQL или другой серверной СУБД.



Важное предупреждение

SQL является стандартизованным языком, но конкретные типы данных, функции и некоторые конструкции зависят от СУБД. Поэтому синтаксис лабораторной работы ориентируется именно на **Access SQL**.

```
CREATE TABLE student (  
    id INTEGER PRIMARY KEY,  
    name TEXT NOT NULL,  
    group_id INTEGER  
);
```

- ▶ имя таблицы задаёт объект схемы;
- ▶ столбцы определяют структуру строки;
- ▶ тип данных ограничивает множество допустимых значений;
- ▶ ограничения задают правила целостности.

DDL: типы данных и ограничения

Элемент	Пример	Идея
Тип	INTEGER, TEXT, DATE	какие значения допустимы
PRIMARY KEY	id	уникальный идентификатор строки
NOT NULL	name	значение обязательно
UNIQUE	email	значения не должны повторяться
CHECK	score BETWEEN 2 AND 5	произвольное условие
FOREIGN KEY	group_id	ссылочная целостность

DDL определяет правила, а не сами экземпляры данных.

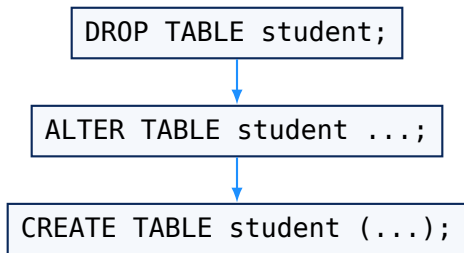
DDL: изменение структуры с помощью ALTER

```
ALTER TABLE student ADD COLUMN email TEXT;
```

```
ALTER TABLE student ADD CONSTRAINT student_email_uq UNIQUE  
(email);
```

- ▶ ADD COLUMN — добавить столбец;
- ▶ ALTER COLUMN — изменить свойства столбца;
- ▶ ADD CONSTRAINT — добавить ограничение;
- ▶ состав конкретных возможностей зависит от СУБД и её версии.

DDL: удаление объектов и осторожность



Операции DDL требуют внимания

Удаление или изменение структуры может сделать существующие запросы и приложения несовместимыми со схемой БД. Поэтому DDL — часть проектирования и сопровождения схемы.

DDL: небольшой пример схемы «Университет»

```
CREATE TABLE student (  
    id INTEGER PRIMARY KEY,  
    name TEXT NOT NULL,  
    group_id INTEGER REFERENCES student_group(id)  
);
```

Что делает этот код?

Он создаёт таблицу и одновременно фиксирует часть логической модели: у студента есть идентификатор, имя и ссылка на группу.

Полная схема должна быть создана в согласованном порядке, учитывая зависимости внешних ключей.

Назначение

DML используется для **работы с данными внутри уже созданных объектов БД.**

INSERT

SELECT

UPDATE

DELETE

Важное различие

DDL меняет **структуру объектов**; DML работает с **экземплярами данных**.

DML: добавление данных — INSERT

```
INSERT INTO Student (ID, Name, GroupID)  
VALUES (101, 'Иванов Иван', 10);
```

- ▶ явно перечислять столбцы — надёжнее, чем полагаться на порядок всех полей;
- ▶ СУБД проверяет типы и ограничения;
- ▶ при нарушении ограничения операция завершается ошибкой.

DML: выборка данных — SELECT

```
SELECT id, name  
FROM Student  
WHERE group_id = 10;
```

- ▶ SELECT определяет, какие значения нужно получить;
- ▶ FROM указывает источник данных;
- ▶ WHERE задаёт условие отбора строк.

Для лабораторной

На этом синтаксисе начинается практическая работа с запросами.

DML: условия выборки — WHERE

Оператор	Пример
Сравнение	<code>score >= 4</code>
И	<code>group_id = 10 AND name IS NOT NULL</code>
Или	<code>group_id = 10 OR group_id = 11</code>
Диапазон	<code>score BETWEEN 3 AND 5</code>
Множество	<code>group_id IN (10,11,12)</code>
Шаблон	<code>Name LIKE 'Ив*'</code>
Пустое значение	<code>email IS NULL</code>

Важно: NULL не равен обычному значению и проверяется через `IS NULL/IS NOT NULL`.

```
UPDATE student  
SET name = 'Иванов И. И.'  
WHERE ID = 101;
```

Критически важно

Запрос UPDATE без корректного условия WHERE может изменить **все строки** таблицы.

То же правило касается и других массовых операций над данными.

DML: удаление данных — DELETE

```
DELETE FROM Student  
WHERE ID = 101;
```

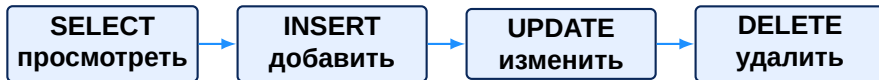
- ▶ удаляются строки, удовлетворяющие условию;
- ▶ ограничения внешних ключей могут запретить удаление;
- ▶ удаление данных и удаление таблицы — разные операции.

Сравнение

DELETE — работа с данными.

DROP TABLE — удаление объекта схемы.

SELECT, INSERT, UPDATE, DELETE: единый цикл работы

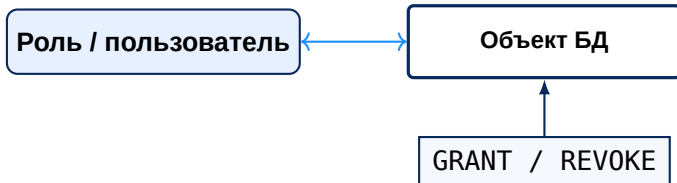


Практическая схема

Создали структуру через DDL → заполнили и изменяем её через DML → доступ ограничиваем через DCL → согласованность операций управляем через TCL.

Назначение

DCL используется для управления **правами доступа** пользователей и ролей к объектам базы данных.



DCL: GRANT — выдать право

```
GRANT SELECT ON student T0 analyst;
```

```
GRANT SELECT, INSERT ON student T0 operator;
```

- ▶ право задаётся для конкретного пользователя или группы;
- ▶ можно разрешать отдельные операции;
- ▶ принцип наименьших привилегий: выдавать только необходимые права.

```
REVOKE INSERT ON student FROM analyst;
```

- ▶ GRANT добавляет разрешение;
- ▶ REVOKE отзывает разрешение;
- ▶ в Access права могут назначаться пользователям и группам в системе безопасности базы данных.

Зачем это нужно

Даже корректно спроектированная БД небезопасна, если любой пользователь может изменять любые данные.

DCI: пример разделения доступа

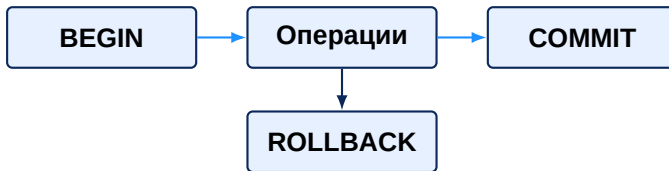
Роль	Право	Назначение
analyst operator	SELECT SELECT, INSERT, UPDATE	анализ данных без изменения операционная работа
auditor admin	SELECT расширенный доступ	контроль и аудит администрирование

Принцип

Права должны отражать реальную роль человека или сервиса, а не выдаваться «на всякий случай».

Транзакция

Транзакция — логически связанная последовательность операций, которую СУБД рассматривает как единое целое для фиксации или отмены результата.



TCL: COMMIT и ROLLBACK

```
BEGIN;  
UPDATE Student SET GroupID = 11 WHERE ID = 101;  
COMMIT;
```

```
BEGIN;  
UPDATE Student SET GroupID = 99 WHERE ID = 101;  
ROLLBACK;
```

- ▶ COMMIT фиксирует изменения транзакции;
- ▶ ROLLBACK отменяет незафиксированные изменения транзакции.

TCL в Access: BEGIN TRANSACTION, COMMIT, ROLLBACK

```
BEGIN TRANSACTION;  
UPDATE Student SET GroupID = 11 WHERE ID = 101;  
COMMIT TRANSACTION;
```

```
BEGIN TRANSACTION;  
UPDATE Student SET GroupID = 99 WHERE ID = 101;  
ROLLBACK TRANSACTION;
```

- ▶ BEGIN TRANSACTION начинает явную транзакцию;
- ▶ COMMIT TRANSACTION фиксирует выполненные изменения;
- ▶ ROLLBACK TRANSACTION отменяет действия текущей транзакции.

Для курса

Подробная программная работа с транзакциями через DAO/ADO рассматривается только как дополнительный материал; на лабораторной основной акцент — на SQL-запросах.

Транзакция на примере перевода



Почему нужна транзакция?

Если первая операция выполнена, а вторая завершилась ошибкой, состояние «деньги списали, но не зачислили» недопустимо. Группа операций должна быть согласована как единое целое.

ACID: почему транзакции являются основой надёжности

Свойство	Смысл
Atomicity	транзакция выполняется целиком или не даёт своего результата
Consistency	после успешной транзакции должны сохраняться заданные правила целостности
Isolation	параллельные транзакции должны взаимодействовать контролируемым образом
Durability	зафиксированный результат должен сохраняться при штатных сбоях

Детали уровней изоляции, блокировок и многоверсионного контроля — отдельная тема; здесь фиксируется только логическая основа.

DDL, DML, DCL и TCL: не путать назначение

Группа	Вопрос	Примеры
DDL DML	Что существует в БД? Какие данные храним и изменяем?	CREATE, ALTER, DROP SELECT, INSERT, UPDATE, DELETE
DCL TCL	Кто что может делать? Как зафиксировать или отменить группу операций?	GRANT, REVOKE BEGIN TRANSACTION, COMMIT, ROLLBACK

Структура → Данные → Доступ → Согласованность

Минимальный сценарий работы с Microsoft Access 2016

1. DDL

создать схему

2. DML

записать данные

3. DCL

выдать права

4. TCL

управлять операциями

Пример рабочего цикла

Создать таблицы → вставить данные → выполнить SELECT → изменить данные в транзакции → зафиксировать результат.

Следующий шаг

В лабораторной работе этот цикл превращается в практику: студенты начинают писать и выполнять реальные SQL-запросы к Microsoft Access 2016.

- ▶ путать таблицу с базой данных, а SQL — с самой СУБД;
- ▶ менять структуру таблиц, не учитывая существующие зависимости;
- ▶ выполнять UPDATE или DELETE без корректного WHERE;
- ▶ выдавать роли избыточные права;
- ▶ выполнять несколько связанных изменений без транзакции;
- ▶ считать, что успешный запуск команды автоматически означает корректность результата.

1. СУБД управляет не только хранением, но и запросами, целостностью, доступом и транзакциями.
2. SQL является основным языком взаимодействия с реляционной СУБД.
3. DDL описывает структуру объектов базы данных.
4. DML выполняет операции над данными.
5. DCL управляет правами доступа.
6. TCL управляет фиксацией и отменой транзакций.
7. Одна прикладная задача часто требует совместного использования всех этих средств.

Контрольные вопросы

1. Какую основную проблему решает СУБД, а не просто набор файлов?
2. Какие основные компоненты СУБД участвуют в обработке запроса?
3. Как Access 2016 выполняет SQL-запрос через ядро СУБД?
4. Что такое SQL и почему его называют декларативным?
5. Чем отличается DDL от DML?
6. В чём разница между DELETE и DROP TABLE?
7. Для чего используются GRANT и REVOKE?
8. Что такое транзакция и зачем нужны COMMIT и ROLLBACK?
9. Чем BEGIN TRANSACTION, COMMIT и ROLLBACK отличаются в Access?
10. Почему отсутствие корректного WHERE опасно в UPDATE и DELETE?

Перед началом работы

Необходимо понимать назначение четырёх групп SQL-операций, но основной практический акцент будет сделан на запросах к таблицам в Microsoft Access 2016.

DDL	создать таблицы и ограничения
DML	вставить, выбрать, изменить и удалить строки
DCL	назначить права ролям при необходимости
TCL	объединить связанные изменения в транзакцию

Лабораторная работа: практическое выполнение SQL-запросов в Microsoft Access 2016.

Спасибо за внимание!